

CryptoQuant Data Engineer 사전 과제 보고서

Bitcoin Velocity 지표 파이프라인 설계 및 Ethereum 로그 수집 파이프라인 구축

문서 목적

과제 1의 정책 기반 Bitcoin Velocity 설계와 과제 2의 Ethereum 로그 수집 구현을, 코드·테스트·Airflow 로그·Delta/DuckDB 직접 조회·노트북 저장 출력·UI 스크린샷으로 연결한 제출용 총괄 보고서임. 검증하지 않은 범위는 상태 라벨로 분리함.

항목	기록 값
지원자	박 종 우
Repository	Private GitHub Repository Collaborator 초대 완료.
제출 일자	2026-06-22
보고서 버전	V3.0 - evidence-aligned submission report
실행 환경	Windows host + Docker Compose / Ubuntu 24.04 LTS / Python 3.12.3 / Airflow 2.10.5 UI / Python container / dbt-duckdb / local Delta Lake (delta-rs)
검증 기준	VERIFIED / PARTIALLY VERIFIED / NOT VERIFIED / BLOCKED

※ 비밀 키, RPC endpoint, 토큰, 개인식별 정보는 본 보고서와 Github 저장소 증거에 기록하지 않음.

0. 문서 원칙, 범위 및 제출 판정

이 보고서는 주장과 증거를 분리한다. 코드·테스트·로그·직접 조회·저장된 노트북 출력 중 하나 이상이 있는 항목만 VERIFIED 로 기록했다. Task 1 은 설계 산출물이고, Task 2 는 로컬 Docker 기반 구현 및 실행 증거가 있는 범위다.

상태	의미	이 보고서의 사용 기준
VERIFIED	코드·테스트·로그·직접 조회 중 하나 이상으로 요구 충족을 확인함.	실행 결과 또는 저장된 출력이 있음.
PARTIALLY VERIFIED	핵심 구조는 있으나 실행 범위 또는 최신성 증거가 제한됨.	남은 범위와 영향도를 함께 기록함.
NOT VERIFIED	설계 또는 코드가 있으나 요구 수준의 실행 증빙이 없음.	완료처럼 표현하지 않음.
BLOCKED	외부 환경 또는 접근 제약으로 확인 불가.	원인·우회안·재개 조건을 기록함.

0.1 Executive Summary

- Task 1 은 CryptoQuant production metric 복제가 아니라, 제공된 Delta Lake 원천 테이블을 전제로 한 assignment-scoped Bitcoin Velocity V1 설계다. 분자·분모·정책·365 일 window·reorg 회복 범위를 버전 가능한 계약으로 분리했다.
- Task 2 는 Airflow `@hourly` DAG → finalized block range → `eth\_getLogs` 10-block chunk → normalization → Delta insert-if-not-exists → dbt selector build → ERC-20 Transfer 및 Tether Treasury flow 순서로 구현됐다.
- 실제 외부 RPC scheduled run 증거는 성공 반환값 33 건이다. 최신 성공 run 은 UTC 2026-06-22 08:00~09:00 이며 raw 169,451 건, inserted 169,451 건, `row\_count\_after=6,082,932`, `dbt.returncode=0`을 기록했다.
- Delta direct inspection 은 `ethereum\_logs\_v2` version 36, file 37 개, row 6,082,932, natural key duplicate 0 건을 기록했다. DuckDB downstream 은 `erc20\_transfers=5,400,325`, `tether\_treasury\_flow=2`, quality summary=1 이다.
- canonical reorg replacement, full-history backfill, provider SLA-alerting-secret rotation 은 구현 또는 운영 검증 범위 밖으로 남겼다. 주피터 노트북의 구 raw schema 는 별도 `PARTIALLY VERIFIED` 증거로 분리했다.

0.2 포지션 정합성 및 증빙 전략

채용 관점	과제에서 보인 구현·설계	주요 증거	상태
온체인 데이터 모델	BTC UTXO 기반 policy metric 과 ETH event log-ERC-20 decode 를 별도 계약으로 정의.	Task 1 data contract, normalizer, dbt model design, fixture decode.	VERIFIED
안정적 수집·적재	UTC interval→block range, 10-block chunk, retry/split, Delta natural key insert-if-not-exists.	Python modules, pytest, notebook 03, scheduled task log.	VERIFIED
장애 대응·재처리	finality guard, Airflow retry, data interval/conf window, duplicate replay 차단.	attempt 1/2 retry→attempt 3 success, retry tests, replay output.	VERIFIED
Lake·변환 모델링	Delta raw schema/partition 및 dbt incremental `erc20_transfers`, `tether_treasury_flow`.	fixture dbt build PASS=43, Delta/DuckDB direct inspection.	VERIFIED
품질·관측성	raw duplicate, invalid log, row count, dbt result 및 analytics counts 를 structured log 와 checks 로 연결.	quality checks, task return payload, validation evidence.	VERIFIED
성능·비용 trade-off	무료 provider 호환을 위해 10-block range, 4 RPS 이하, concurrency 1 을 강제.	config/chunking invariant, incident history.	VERIFIED
AI 활용 통제	AI 는 범위 분해·리팩토링·테스트 후보·문서 정합성에 사용. 최종 반영은 실행/코드 대조 후 결정.	AI transparency document, test/build/log references.	VERIFIED

제출 경계

로컬 `data/delta`, `data/analytics`, `airflow/logs`, `dbt/target`은 runtime artifact 다. 증거 인용을 위해 경로를 기록하지만 source file 자체와 동일시하지 않는다. Private Repository 최종 push 및 Collaborator 초대는 repository metadata 외부에서 확인해야 하므로 이 보고서에서는 PARTIALLY VERIFIED 로 두었음.

1. 과제 요구사항 추적 매트릭스

요구사항을 설계·구현·실행 증거 단위로 분해했다. 상태는 제출 시점에 다시 갱신해야 하는 Repository 권한 항목과, 현재 검증되지 않은 production 범위를 포함한다.

구분	요구사항	핵심 구현 / 정책	증빙 위치	상태
공통	AI 활용·검증 기록	대표 프롬프트, 사람의 결정, 반영 전 검증을 분리.	Section 2; docs/09; docs/10.	VERIFIED
공통	Private GitHub upload 및 Collaborator	제출 경계와 체크리스트에 반영. repository 내부 evidence 만으로 remote 접근 상태는 확인 불가.	Section 7 checklist.	PARTIALLY VERIFIED
공통	README 실행 방법·설계 근거	README 가 setup, execution, data contract, limitation, evidence path 를 제공.	README.md; docs/04.	VERIFIED
Task 1	Velocity 필드 명세·선정 근거	block/tx/tx_input/tx_output/utxo 최소 계약과 lifecycle 필요조건 정의.	Section 3.2; Task1 docs 03.	VERIFIED
Task 1	Volume / Supply 정책	gross output volume, policy-eligible supply, dormancy-adjusted variant 을 분리.	Section 3.3; Task1 docs 02-03.	VERIFIED
Task 1	SQL 또는 pseudocode-dummy output	Best-chain snapshot, daily volume/supply, 365-day date spine SQL 및 3-day fixture.	Section 3.4; Task1 docs 03.	VERIFIED
Task 1	Daily batch-reprocess	same DAG path, policy confirmation, MERGE publish, audit design.	Section 3.5-3.6; Task1 docs 04.	VERIFIED
Task 1	Chain reorganization	common ancestor, affected range, audit preservation, bounded recomputation design.	Section 3.7; Task1 docs 05.	VERIFIED
Task 2	Hourly `eth_getLogs` Airflow DAG	`ethereum_hourly_logs`, `@hourly`, one `run_interval` task.	airflow/dags; UI screenshots; task logs.	VERIFIED
Task 2	Block range-backfill-idempotency	[start,end)→inclusive block range; conf/data interval; natural key dedup.	block_range.py; delta_writer.py; notebook 03.	VERIFIED

구분	요구사항	핵심 구현 / 정책		증빙 위치	상태
Task 2	Retry / failure recovery	finality retry, 429/5xx/timeout retry, split-on-range/result limits.		rpc_client.py; block_range.py; task log attempt history.	VERIFIED
Task 2	Delta Lake schema / partition / incremental write	`block_date_utc` partition; raw v2 table; insert-only new keys.		delta_writer.py; docs/02; Delta inspection.	VERIFIED
Task 2	dbt `ethereum_logs` → `erc20_transfers`	Transfer topic decode, lower-case addresses, uint256 policy, incremental model.		dbt model design; fixture build; notebook 03.	VERIFIED
Task 2	dbt `erc20_transfers` → Treasury flow	USDT contract + Treasury address, hourly/direction flow aggregation.		dbt model design; build; DuckDB `flow=2`.	VERIFIED
Bonus	New dbt model without DAG edit	`tag:ethereum_hourly` selector + `ref()` graph brings quality summary into build.		dbt ls; DAG hash check; Section 4.6.	VERIFIED
Limit	Canonical reorg replacement	finality buffer + raw block hash stored; common-ancestor replacement fixture absent.	Section 6; docs/03/05.		NOT VERIFIED

2. AI 도구 활용 및 검증 기록

판정 원칙

AI 를 사용했다는 사실은 구현 검증이 아니다. AI 제안은 후보로만 취급하고,
코드·테스트·dbt build·Airflow 로그·Delta/DuckDB 직접 조회·Markdown link check 중 하나 이상으로 확인한 경우에만 반영했다.

2.1 활용 원칙

항목	작성 내용
활용 목적	요구사항을 구현·증빙·리스크 단위로 분해, Airflow/Delta/dbt 책임 분리, 테스트 및 edge case 후보 탐색, 코드·SQL·문서 정합성 점검.
사람의 결정 범위	Bitcoin Velocity policy, natural key, retry/backfill 경계, Delta/schema 전략, dbt model grain, 상태 라벨과 제출 범위는 코드·실행 증거를 바탕으로 제출자가 최종 결정.
검증 기준	`compileall`, ruff, pytest, fixture `dbt build`, DagBag import, Airflow task log, Delta/DuckDB direct inspection, notebook saved output, Markdown link check.
제외 규칙	secret 입력·출력 금지, 근거 없는 수치·사실 인용 금지, 실행하지 않은 결과를 완료로 표시하지 않음, production SLA·full-history backfill 을 과제 실행 증거로 확장하지 않음.

2.2 대표 활용 이력

No.	도구	사용 목적	프롬프트 / 지시 요약	검증 방식	결정
1	GPT	요구사항·증빙 분해	Repository 의 Python·SQL/dbt·Markdown·체크리스트를 함께 정합화하라는 지시.	requirement matrix, compileall, pytest, dbt build, link check.	반영: 누락 방지 및 상태 분리.
2	Codex	코드·SQL 구조화	Airflow, block range, RPC, Delta writer, dbt runner 책임을 분리하고 설명 주석을 강화.	ruff, targeted/full pytest, DagBag import, fixture build.	부분 반영: environment-dependent 영역은 제한으로 기록.
3	GPT/Codex	AI 투명성 문서화	AI 가 확장한 검토 범위와 제출자가 책임진 최종 판단을 분리.	README, docs/09, traceability matrix 대조.	반영: 대표 프롬프트 5 개만 요약.

No.	도구	사용 목적	프롬프트 / 지시 요약	검증 방식	결정
4	Codex	노트북 검증 보조	notebooks 의 source code/data validation focus 를 정리하고 saved output 을 보존.	nbclient execution, notebook JSON parse, output review.	반영: 03 은 VERIFIED, 04 는 PARTIALLY VERIFIED.
5	GPT/Codex	image evidence 연결	Airflow UI screenshot 을 실행 증거 범위와 한계에 연결.	이미지 수동 판독 + task log/Delta/DuckDB 대조.	반영: UI 만으로 data correctness 를 주장하지 않음.

2.3 AI 제안 검증 로그 요약

검증 대상	AI 제안	검증 결과	조치
요구사항 해석	Task 1 과 Task 2 를 동일한 구현 과제로 취급하지 않도록 분리.	Task 1 은 설계·의사 SQL; Task 2 는 실행 코드/증거로 확인.	반영.
Airflow DAG	data interval 기반 backfill, finality guard, retry 확인.	DagBag import 와 retry 후 success log 가 존재.	반영.
Delta Lake	natural key 와 schema contract 를 점검.	v2 Delta row/duplicate 는 VERIFIED. base Delta schema 는 stale.	v2 기준으로 주장, base path 는 partial.
dbt model	selector/ref graph 로 새 모델 자동 포함.	`dbt ls`와 fixture build 에서 quality summary 포함.	반영.
문서·주석	코드/문서상 용어와 상태 불일치 탐지.	Markdown local links ok; traceability origin matrix 기록.	반영.

3. 과제 1 - Bitcoin Velocity 지표 파이프라인 설계

Task 1 은 실행 가능한 Bitcoin ETL 을 주장하지 않는다. 제공된 {`block`, `tx`, `tx\_input`, `tx\_output`, `utxo`} Delta Lake 테이블을 전제로, 재현 가능한 정책 기반 지표 계약·배치·품질·reorg 복구를 설계한 산출물이다.

3.1 문제 이해, 범위 및 핵심 가정

항목	정의 또는 결정	근거 / 영향
지표 정의	<code>`assignment_velocity_365d_policy_eligible_utxo_v1` = trailing 365-day gross on-chain output volume / day-end policy-eligible UTXO supply`.</code>	공개 제품의 내부 estimated-volume 규칙은 제공되지 않았으므로 과제 전용 정책을 별도 정의.
산출 주기	UTC 일 단위 publish. 분자는 최근 365 calendar days, 분모는 기준일 day-end supply.	daily cadence 와 365-day volume window 를 분리.
입력 원천	<code>`block`, `tx`, `tx_input`, `tx_output`, `utxo`</code> Delta Lake; historical lifecycle 재구성 전제.	현재 UTXO snapshot 만으로 과거 day-end supply 를 복원하지 않음.
출력 grain	<code>`metric_date` + `metric_variant` + `metric_contract_version` 1 행.</code>	정책 변경·recalc 이 섞이지 않도록 logical key 로 MERGE.
확정성	Best Chain snapshot, <code>`chain_revision_id`</code> , cutoff block, required successor blocks 기록.	Bitcoin 은 deterministic finality 가 없으므로 confirmed-by-policy 를 명시.
범위 제외	오프체인 거래량, entity ownership, exchange internal trade, CryptoQuant proprietary estimation method.	원천 사실과 정책/추정을 분리하고 과장 방지.

핵심 정책

분자는 non-coinbase, Best Chain, non-provably-unspendable output 의 gross output value 다. Change output 과 self-churn 을 제거하지 않으므로 economic transfer volume 으로 명명하지 않는다. 기본 분모는 day-end policy-eligible UTXO supply 이며, dormant UTXO 는 lost coin 으로 단정하지 않고 보조 민감도 지표에서만 분리함.

3.2 Velocity 계산용 최소 데이터 계약

원천	필드	타입 예시	필요한 이유	품질 / 제약
block	block_hash / previous_block_hash	string	Best Chain 연결, common ancestor, reorg 비교.	hash unique; not null.
block	height / block_time / observed_at	bigint / timestamp	UTC date, cutoff, snapshot provenance.	canonical revision 내 height unique; UTC normalize.
tx	txid / block_hash / tx_index / is_coinbase	string / bigint / boolean	블록 귀속, non-coinbase volume filter.	canonical chain consistency.
tx_input	txid / input_index / prev_txid / prev_vout	string / int	consumed UTXO 와 lifecycle 재구성.	coinbase 예만 previous ref nullable.
tx_output	txid / vout / value_sats / script class	string / int / bigint	생성 UTXO 와 gross output volume.	value >= 0; output key unique.
utxo lifecycle	created/spent height, is_spent, value_sats	bigint / boolean	day-end supply 와 dormant age policy.	create < spend; revision-consistent.
policy metadata	definition/policy version, chain_revision_id	string	같은 날짜라도 정책·체인 상태 차이를 분리.	immutable versioned contract.

3.3 Transaction Volume 및 Circulating Supply 정책

정책 항목	선택한 정의	왜 이 정의인가	반례 / 한계	버전
Transaction Volume	Best Chain 의 non-coinbase / non-unspendable output value 합계.	원천 온체인 자료만으로 재현 가능.	change output-self-churn 포함 가능.	gross_output_v1
Coinbase	분자에서 제외. 분모는 100-block maturity 통과분만 포함.	일반 전송 활동과 발행/보상을 분리.	maturity 경계는 cutoff height 기준.	policy_v1
Change output	V1 에서는 별도 heuristic 으로 제거하지 않음.	entity/address clustering 이 제공되지 않음.	경제적 이동량보다 과대 계상 가능.	gross_output_v1

정책 항목	선택한 정의	왜 이 정의인가	반례 / 한계	버전
Basic Supply	Best Chain day-end unspent, non-unspendable, matured UTXO sum.	정책을 숨기지 않고 설명 가능.	protocol total supply 와 동일하지 않음.	eligible_utxo_v1
Dormant UTXO	기본 분모에는 포함. 10 년 예시 threshold 는 보조 sensitivity variant 에서만 차감.	장기 미사용은 lost proof 가 아님.	threshold 는 정책 파라미터.	dormant_10y_v1
Burn / loss	script 수준에서 provably unspendable 인 output 만 제외.	외부 label 만으로 burn/loss 를 단정하지 않음.	burn registry 는 future extension.	burn_script_v1
정책 변경	`effective_from_date` + immutable contract version.	과거 결과 재현과 version mixing 방지.	policy change 는 time series break 를 유발 가능.	metric_contract_v1

3.4 계산 로직, 의사 SQL 및 더미 출력

```
-- assignment-specific pseudocode: 365-day Velocity
WITH date_spine AS (
  SELECT metric_date FROM dim.calendar
  WHERE metric_date BETWEEN :start_date AND :end_date
), daily_base AS (
  SELECT d.metric_date, v.daily_gross_volume_btc, s.policy_eligible_supply_btc
  FROM date_spine d
  LEFT JOIN silver.daily_volume v USING (metric_date)
  LEFT JOIN silver.daily_supply s USING (metric_date)
), windowed AS (
  SELECT metric_date,
    SUM(COALESCE(daily_gross_volume_btc, 0.0))
      OVER (ORDER BY metric_date ROWS BETWEEN 364 PRECEDING AND CURRENT ROW) AS
volume_365d,
    COUNT(daily_gross_volume_btc)
      OVER (ORDER BY metric_date ROWS BETWEEN 364 PRECEDING AND CURRENT ROW) AS
volume_days,
    COUNT(policy_eligible_supply_btc)
      OVER (ORDER BY metric_date ROWS BETWEEN 364 PRECEDING AND CURRENT ROW) AS
supply_days,
    policy_eligible_supply_btc
  FROM daily_base
)
SELECT metric_date, volume_365d / policy_eligible_supply_btc AS velocity
FROM windowed
WHERE volume_days=365 AND supply_days=365 AND policy_eligible_supply_btc > 0;
```

metric_date	normal output sum	illustrative rolling volume	policy-eligible supply	illustrative velocity	검증 메모
2026-06-01	100.0 BTC	100.0 BTC	100.0 BTC	1.000	3-day 축소 fixture.
2026-06-02	98.0 BTC	198.0 BTC	100.0 BTC	1.980	gross volume 은 supply 증가분이 아님.
2026-06-03	94.5 BTC	292.5 BTC	100.0 BTC	2.925	(100+98+94.5)/100.

더미 fixture 는 계산 경로와 단위-0 division-date spine 규칙을 설명하기 위한 설계 예시다. 실제 Bitcoin DB ETL 또는 SQL 실행 결과가 아니므로 실행 성과로 표시하지 않음.

3.5 일 단위 배치, 품질, 역등성 및 Reorg 설계

순서	Task	입력	출력	실패 / 재처리
1	resolve_data_interval	Airflow data interval	metric_date range	설정 오류 hard fail.
2	resolve_chain_checkpoint	node/raw snapshot	Best Chain tip, cutoff, revision	재시도 후 hard fail.
3	validate_chain_completeness	block snapshot	height/parent hash validation	문제 날짜 publish 보류.
4	build_utxo_lifecycle	tx_input/output + Best Chain	revision lifecycle	hard fail; same path rerun.
5	build_daily_components	lifecycle + output	daily volume / supply	partition recompute.
6	calculate_velocity	365-day date spine	metric staging	coverage 부족 publish 중단.
7	run_quality_gates	staging	quality status	hard fail 또는 review alert.
8	publish_metric	quality-passed rows	Gold Delta MERGE	transaction retry; audit append.

3.6 품질과 관측성

검증 항목	검증 규칙	실패 시 조치	증빙 / 설계 위치
Best Chain continuity	height gap 또는 parent hash mismatch 없음.	publish stop, source re-sync.	Task1 docs 04.
Best Chain uniqueness	same revision + height 는 block 1 건.	hard fail.	Task1 docs 04.
UTXO lifecycle	spend height > create height; output key unique.	hard fail, lifecycle rebuild.	Task1 docs 03-04.
Supply validity	supply non-null, >0, policy predicate 만족.	quarantine + review.	Task1 docs 04.

검증 항목	검증 규칙	실패 시 조치	증빙 / 설계 위치
Rolling coverage	calendar / volume / supply 365 days 모두 충족.	publish stop; missing source backfill.	Task1 docs 03.
Review alert	volume/supply/dormant-spent 급변.	alert + analyst review; hard fail 아님.	Task1 docs 04.

3.7 Chain Reorganization 대응

구분	설계 결정	구체 동작	현재 검증 상태
감지	stored hash 와 current Best Chain hash 비교.	mismatch 시 previous hash 를 역추적하여 common ancestor 탐색.	VERIFIED - design
영향 범위	common ancestor 다음 block 의 UTC date 부터 latest confirmed date.	volume 뿐 아니라 UTXO day-end supply 도 fork 이후 변경될 수 있어 넓게 재계산.	VERIFIED - design
원천 정정	새 branch 를 새 `chain_revision_id`로 저장.	old branch 를 물리 삭제하지 않고 audit 에 superseded 상태 보존.	VERIFIED - design
파생 정정	affected start 부터 current confirmed 까지 components/velocity 재생성.	window 입력은 affected date - 364 일부터 다시 읽고 Gold MERGE.	VERIFIED - design
공개 기준	policy confirmation 을 통과한 current Best Chain 만 Gold publish.	pending/superseded 는 audit history 로 분리.	VERIFIED - design
실행 fixture	Bitcoin reorg simulator / source DB execution.	과제-1 설계 프로젝트에는 실제 구현·실행 증거 없음.	NOT VERIFIED

Task 1 결론

Task 1 은 “지표 수치”보다 재현 가능한 의미 계약을 먼저 고정한 설계다. chain state, policy version, data completeness, reorg audit 을 결과 메타데이터에 묶어, 동일한 입력·정책에서는 같은 결과가 수렴하고 체인 상태 변경에는 영향 범위를 추적해 재계산하는 구조를 정의했다. 실제 Bitcoin ETL 실행은 NOT VERIFIED 임.

4. 과제 2 - Ethereum 로그 수집 파이프라인 구축

Task 2 는 실제 코드와 로컬 Docker 실행 증거가 있는 구현 범위다. 기본 acceptance scope 는 Ethereum mainnet 의 `Transfer(address,address,uint256)` topic0 를 모든 address 대상으로 수집하고, dbt 에서 configured USDT contract 와 Treasury address 기준으로 downstream flow 를 계산한다.

4.1 전체 아키텍처 및 실행 환경

```
Airflow @hourly data_interval
-> run_interval(start, end)
-> finalized head check
-> timestamp -> [from_block, to_block] binary search
-> Transfer topic0 eth_getLogs (chunks <= 10 blocks)
-> normalize + Delta insert-if-not-exists
-> dbt build --select tag:ethereum_hourly --vars window
-> erc20_transfers -> tether_treasury_flow -> quality summary
```

구성 요소	선택	책임	선택 근거
Orchestrator	Airflow in Docker Compose; DAG `ethereum_hourly_logs`.	UTC hour interval, retry, run identity, backfill-compatible entry.	시간 계약과 retry lifecycle 을 scheduler 에 위임.
RPC Provider	credential-injected Ethereum JSON-RPC endpoint; Alchemy Free compatible constraints.	`eth_getBlockByNumber`, `eth_getLogs`.	endpoint/key 는 code/log/report 에 남기지 않음.
Collection scope	`transfer_topic_all_addresses` ; chain_id=1; Transfer topic0 only.	수집 중 scope drift 차단.	address/from/to filter 로 축소하지 않아 acceptance scope 를 고정.
Storage	local Delta Lake via delta-rs.	raw normalized logs, partition, natural-key idempotency.	로컬 재현성과 data contract inspection.
Transform	dbt-duckdb.	Delta source, incremental models, dbt graph/test.	model dependency 를 dbt `ref()`에 위임.
Test / evidence	pytest, fixture dbt build, notebook output, Airflow logs/UI, Delta/DuckDB inspection.	정상.retry-replay-schema freshness 경계 확인.	서로 다른 증거가 과장을 교차 억제.

실행 제약과 안전 불변식

`MAX\_BLOCKS\_PER\_LOG\_REQUEST=10`, `ETH\_RPC\_REQUESTS\_PER\_SECOND<=4`, `ETH\_RPC\_CONCURRENCY=1`을 code-level validation 으로 제한했다. 동일 natural key `(chain\_id, transaction\_hash, log\_index)`는 raw event 1 건을 대표하며 replay/backfill 로 row 수가 증가하면 안 됨.

4.2 Airflow DAG

DAG 항목	설계 값	근거 / 불변식
DAG ID	`ethereum_hourly_logs`.	DagBag import 에서 단일 active DAG 로 확인.
schedule	`@hourly`.	Airflow UI DAG home 과 parsed DAG evidence 에서 확인.
task	`run_interval` single task.	Python pipeline 이 block range→collect→Delta→dbt 를 하나의 bounded interval 로 orchestration.
data interval	UTC `[start, end)` half-open.	경계 block 중복/누락 방지; `now()-1h` 기준 사용하지 않음.
manual/backfill	Airflow `data_interval_start/end` 또는 DAG run conf window.	scheduled/manual 이 동일 pipeline path 를 사용. full-history run 은 미실행.
max_active_runs	1.	local single writer 및 DB file lock risk 완화.
retry	finality 미도달 및 transient RPC error 대상.	attempt 1/2 retry, attempt 3 success evidence.

DAG source path: `airflow/dags/ethereum\_hourly\_logs.py`. Airflow UI screenshot 은 DAG 등록·schedule·run history 를 보조 증명하고, 실제 row-level 결과는 task log 및 Delta/DuckDB 산출물과 대조하여 판정함.

4.3 Block Range, Backfill, 멍등성 및 실패 재처리

문제	실제 구현 / 정책	핵심 키 또는 계산	검증 경계
시간 → block range	`eth_getBlockByNumber("finalized")` 후 start/end 각각에 대해 first block at-or-after 를 이진 탐색.	`to_block = end_exclusive_block - 1`; final range 는 inclusive.	block_range unit tests; latest task log range 기록.
finality guard	finalized timestamp < interval end 면 `RetryableIntervalNotFinalized`.	eth_getLogs/Delta/dbt 이전에 중단.	retry 2 회 후 attempt 3 success history.
범위 분할	inclusive range 를 10 block 이하 chunk 로 split.	`chunk_end=min(chunk_start +9,to_block)`.	chunking invariant and config validation.
RPC retry	429, 5xx, timeout, transport/invalid JSON 을 bounded retry.	range/result limit 은 retry 보다 split 우선.	rpc retry tests + incident history.
멍등성	Delta writer 가 existing natural keys 조회 후 new rows 만 insert.	`chain_id + transaction_hash + log_index`.	notebook 03: first insert 1, second insert 0, duplicate 0.
Backfill	logical data interval 또는 conf start/end 를 동일 `run_interval` 경로로 처리.	과거 1 시간 scope 재실행 가능.	구조는 VERIFIED; full-history 비용 run 은 미실행.
부분 실패	retryable chunk 는 split; single block 계속 실패면 explicit failure.	성공 row 를 silent partial success 로 처리하지 않음.	code/design verified; exhaustive fault matrix 는 partial.

4.4 Delta Lake raw layer

컬럼 그룹	대표 컬럼	nullable	설계 근거	비고
Identity	`chain_id`, `transaction_hash`, `log_index`	N	retry/backfill idempotency natural key.	Ethereum mainnet chain_id=1.
Block / reorg	`block_number`, `block_hash`, `transaction_index`, `removed`	N	range ordering, raw provenance, reorg analysis.	canonical replacement 은 미구현.
Raw event	`contract_address`, `topic0..topic3`, `data_raw`	topic1-3/data conditional	원문 보존으로 decode 변경 시 재처리 가능.	address lowercase normalize.

컬럼 그룹	대표 컬럼	nullable	설계 근거	비고
uint256 contract	`data_uint256_decimal_text`, `data_uint256_decode_status`	decimal text Y; status N	large integer float 오차 회피.	numeric overflow 시 status 로 분리.
Time / audit	`block_timestamp_utc`, `block_date_utc`, interval start/end, `ingested_at_utc`	timestamp policy- dependent	UTC time analysis, partition, run replay provenance.	`block_date_utc` partition.

설계 항목	결정	근거
Canonical evidence path	`data/delta/ethereum_logs_v2`.	latest source contract and E2E evidence path; base `ethereum_logs` is stale fixture-like artifact.
Partition	`block_date_utc`.	UTC predicate pushdown, hourly window and late/replay traceability.
Incremental write	insert-if-not-exists by natural key.	replay/backfill duplicate prevention; no blind append.
Schema evolution	current writer contract must match table before interpreting freshness.	notebook 04 detects old schema as PARTIALLY VERIFIED rather than hiding drift.
Reorg handling	finality buffer + raw `block_hash` retention.	common-ancestor canonical replacement is future hardening, not claimed.
Compaction	not demonstrated as production operation.	local assignment scale; file counts and metadata inspected only.

Delta direct-inspection evidence

`ethereum\_logs\_v2`: Delta version 36, file count 37, row count 6,082,932, natural key duplicate 0. 최신 Airflow task payload 의 `row\_count\_after=6,082,932`와 일치. 반면 notebook 04 가 읽은 base `ethereum\_logs`는 version 0 / 1 row / 구 schema 여서 `PARTIALLY VERIFIED`로 분리함.

4.5 dbt incremental 모델링

4.5.1 `ethereum\_logs` → `erc20\_transfers`

항목	구현 / 정책
입력	Delta raw `ethereum_logs` relation 을 DuckDB `delta_scan()`으로 노출.
필터	ERC-20 `Transfer(address,address,uint256)` topic0 와 raw validity/removed policy 를 사용.
디코딩	topic1/from, topic2/to address 를 lower-case 20-byte address 로 변환; data raw uint256 를 decimal text/status 와 함께 유지.
unique key	`chain_id + transaction_hash + log_index`.
USDT amount policy	configured USDT contract row 에만 decimals=6 기준 `amount_usdt`를 채움; float 를 사용하지 않음.
incremental / test	window variables 및 incremental materialization; fixture `dbt build --select tag:ethereum_hourly` PASS=43.
예외	malformed topic/data, non-standard token, uint256 numeric overflow 는 raw/decode status 로 식별; token metadata dimension 은 미구현.

4.5.2 `erc20\_transfers` → `tether\_treasury\_flow`

항목	구현 / 정책
Treasury address	`0x5754284f345afc66a98fbb0a0afe71e0f007b949` (configured env default).
Token scope	Ethereum mainnet configured USDT contract `0xdac17f958d2ee523a2206206994597c13d831ec7`; external address label is configuration, protocol truth 와 구분.
Inflow / Outflow	`to_address = treasury` → INFLOW; `from_address = treasury` → OUTFLOW.
집계 grain	`hour_start_utc + direction` 중심 flow aggregation; quality model 은 현재 window 의 row/count/amount summary.
지표	transfer count, raw/USDT directional amount, net-flow interpretation support.
검증	fixture dbt build PASS=43; live DuckDB direct inspection `tether_treasury_flow=2`, quality summary=1.
제한	token metadata validity dimension, on-chain `decimals()` check, standalone netflow model 은 미구현.

4.6 신규 dbt 모델의 DAG 수정 없는 자동 반영 - Bonus

선택지	선택한 방식	장점	실행 증거
Airflow-dbt integration	DAG 는 개별 model 명을 하드코딩하지 않고 `run_dbt_build()`에서 `dbt build --select tag:ethereum_hourly` 수행.	새 모델은 dbt tag/ref graph 에 연결하면 DAG Python 변경 없이 선택 대상이 됨.	DAG file hash unchanged; active DAG source has no model names.
New model	`tether_treasury_flow_quality_summary` with `ref("tether_treasury_flow")`.	dummy model 이 아니라 current window 의 flow row count-direction count-amount summary 제공.	`dbt ls --select tag:ethereum_hourly` 에 4 models 와 quality model 포함.
Trade-off	single dbt task (not dynamic task mapping).	구현 단순성·재현성·dbt lineage 를 활용.	모델별 Airflow task observability 는 낮음.

Bonus 판정: VERIFIED

신규 quality summary model 을 실제로 추가한 뒤, Airflow DAG 파일 hash 가 불변인 상태에서 `tag:ethereum\_hourly` selector 와 `ref()` dependency 로 build selection 에 포함된 것을 확인했다. Airflow dynamic task mapping 을 구현했다는 주장은 하지 않음.

5. 테스트, 검증 결과 및 데이터 품질 기준

구분	테스트 시나리오	기대 결과	실제 결과	상태 / 증빙
Static	Python compile / Docker config / ruff	syntax, import, compose config clean.	`compileall` pass; compose config exit 0; ruff all checks passed.	VERIFIED
Unit	UTC interval → block range	boundary overlap/loss 없음; finalized guard.	block range tests; code enforces [start,end) → inclusive range.	VERIFIED
Unit	RPC transient error	429/5xx/timeout retry; range/result issue split.	retry test suite and runtime retry logs.	VERIFIED
Unit	ERC-20 topic/data decode	from/to/value match fixture.	notebook 03 decoded one transfer, expected fixture matched.	VERIFIED
Integration	same interval replay	second run inserts 0 new natural keys.	notebook 03: first 1; second 0; duplicate 0.	VERIFIED
Integration	fixture dbt build	models/tests build with no error.	PASS=43 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=43.	VERIFIED
Integration	full pytest	test suite pass.	`python -m pytest -q`: 49 passed in workspace-dev.	VERIFIED
E2E	external RPC scheduled hourly collection	raw→Delta→dbt success return.	33 successful scheduled return values; latest payload recorded.	VERIFIED
Data contract	base accumulated raw freshness	current Python schema and current staging relation.	notebook 04 detects old base schema/legacy relation.	PARTIALLY VERIFIED
Reorg	canonical replacement fixture	affected partition recompute + audit.	not implemented; finality buffer only.	NOT VERIFIED

5.1 실행 증빙 목록

ID	무엇을 증명하는가	파일 / 명령 / 경로	검토 결과
E-01	DAG parse-schedule-task ID	DagBag import: `import_errors={}`, DAG `ethereum_hourly_logs`, schedule `@hourly`, task `run_interval`.	VERIFIED

ID	무엇을 증명하는가	파일 / 명령 / 경로	검토 결과
E-02	외부 RPC 1 시간 scheduled run	Airflow logs: 33 successful scheduled return values; latest scheduled run 2026-06-22 08:00Z.	VERIFIED
E-03	Delta schema / row count	`data/delta/ethereum_logs_v2`: version 36, file 37, row 6,082,932.	VERIFIED
E-04	replay idempotency	`03_fixture_etl_replay_idempotency_validation.ipynb` saved output.	VERIFIED
E-05	dbt model/test build	fixture `dbt build --select tag:ethereum_hourly --no-partial-parse`.	VERIFIED
E-06	Treasury downstream materialization	`ethereum_analytics_v2.duckdb`: erc20 5,400,325; flow 2; quality 1.	VERIFIED
E-07	Airflow UI registration/history	`data/imgs/task_02_01..04_image.png`.	PARTIALLY VERIFIED
E-08	base data freshness drift surfaced	`04_accumulated_pipeline_data_freshness_validation.ipynb` output.	PARTIALLY VERIFIED

5.2 최신 E2E task log 과 storage 대조

```
latest successful scheduled run
run_id      = scheduled__2026-06-22T08:00:00+00:00
window      = [2026-06-22T08:00:00Z, 2026-06-22T09:00:00Z)
block_range = [25371803, 25372102]
raw         = 169451
normalized  = 169451
invalid     = 0
inserted    = 169451
duplicate_skipped = 0
row_count_after = 6082932
dbt.returncode = 0
```

대상	직접 조회 결과	해석
Airflow scheduled logs	successful scheduled return=33; raw sum=5,411,781; inserted sum=5,411,781.	여러 scheduled intervals 의 raw→dbt success 경로 확인.

대상	직접 조회 결과	해석
Delta v2	version=36; file_count=37; row_count=6,082,932; duplicate natural key=0.	latest task payload row count 와 일치.
DuckDB v2	`erc20_transfers`=5,400,325`; `tether_treasury_flow`=2`; `quality_summary`=1`.	downstream relation materialization 존재.
Base Delta / notebook 04	version=0; old schema fields; 1 row; `stg_ethereum_logs` legacy relation.	current code contract evidence 로 사용하지 않고 partial drift evidence 로 유지.

5.3 노트북 저장 출력과 Airflow UI 이미지 증거

노트북 증거 경계

노트북	목적	저장된 핵심 출력	판정
01 RPC connection smoke	credential-injected provider 연결 smoke.	저장된 실행 output 없음. credential 제공 시 수행하는 보조 노트북.	BLOCKED / not executed
02 eth_getLogs sample	Transfer sample 및 decoder 확인.	저장된 실행 output 없음. 외부 RPC 비용/credential 경계.	BLOCKED / not executed
03 fixture replay idempotency	fixture→normalize→decoder→Delta replay.	raw=2, natural key unique=1, normalized=2, invalid=0, decoded=1, first insert=1, second insert=0, duplicate=0.	VERIFIED
04 accumulated freshness	current code contract vs base Delta/DuckDB 비교.	base raw schema drift; current `ethereum_logs` relation missing; downstream relations exist.	PARTIALLY VERIFIED

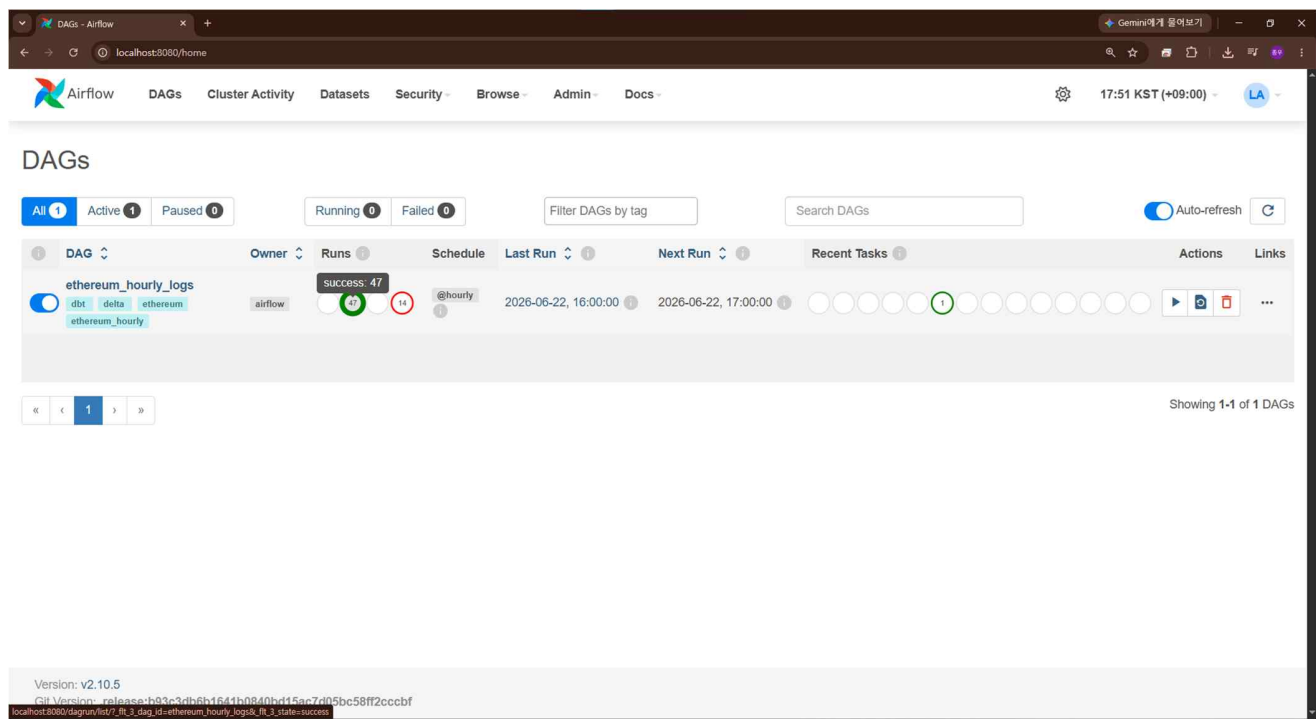
왜 WsrcWnotebooksW02 번 과제 04_accumulated_pipeline_data_freshness_validation.ipynb 를 실패로 숨기지 않았는가
현재 canonical E2E evidence 는 `ethereum\_logs\_v2`와 `ethereum\_analytics\_v2.duckdb`다. 그러나 04 번 노트북은 base path 가 구 schema/legacy relation 을 가진다는 사실을 드러낸다. 따라서 최신 v2 성공 증거와 base accumulated artifact 의 stale 상태를 서로 분리하는 것이 정합한 해석임.

Airflow UI image evidence 의 해석

이미지	화면 / 관측	증거로 볼 수 있는 범위	한계
task_02_01	DAGs home: `ethereum_hourly_logs`, `@hourly`, tags, success/failed run history, Airflow 2.10.5.	DAG registration 및 hourly schedule UI 존재.	task log payload-row-level correctness 를 단독 증명하지 않음.
task_02_02	DAG grid: displayed runs 61, success 47, failed 14.	manual/scheduled history 가 존재하고 failure 를 숨기지 않음.	실패 원인과 latest code/data contract 는 UI 단독 판정 불가.
task_02_03	failed task instance list: `run_interval` failed records 13.	디버깅 대상과 retry/failure history 가 보존됨.	exception detail 은 task log 필요.

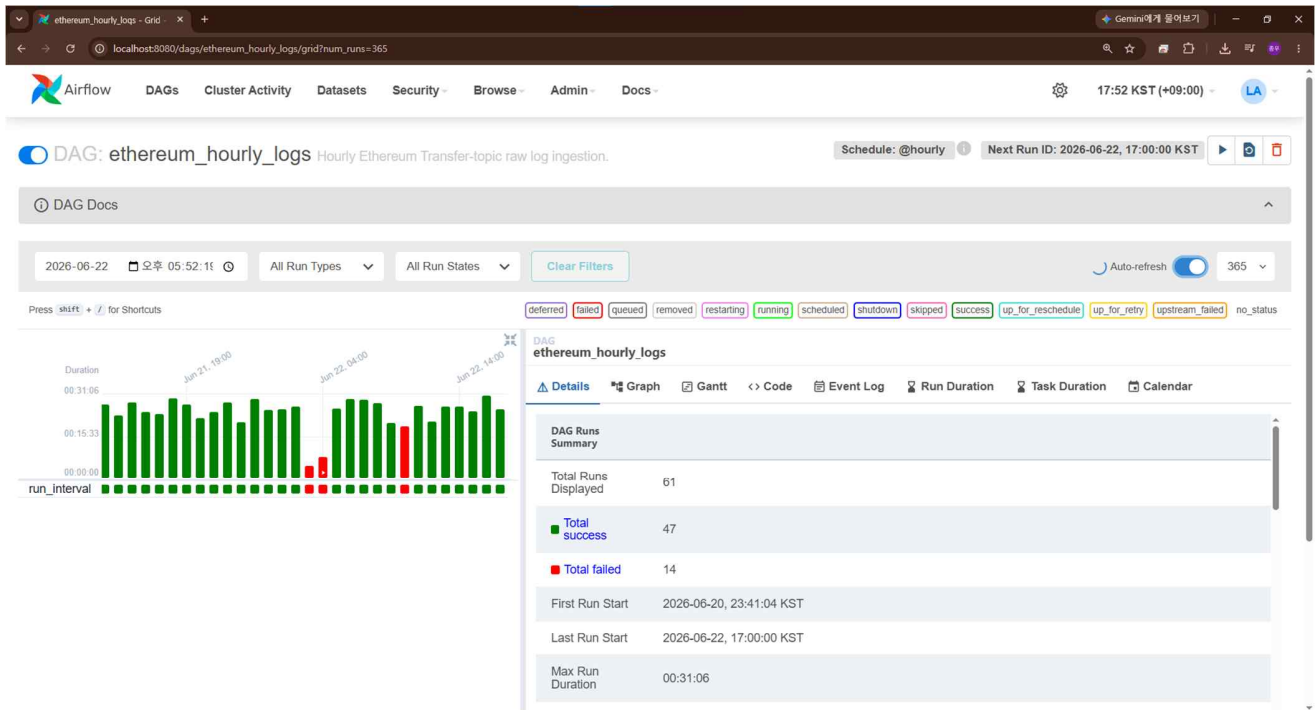
이미지	화면 / 관측	증거로 볼 수 있는 범위	한계
task_02_04	success DAG run list: success records 47, scheduled run list.	UI 기준 success history 존재.	success 가 최신 schema/commit 과 일치하는지는 Delta/log 대조 필요.

Figure 1. Airflow DAGs home



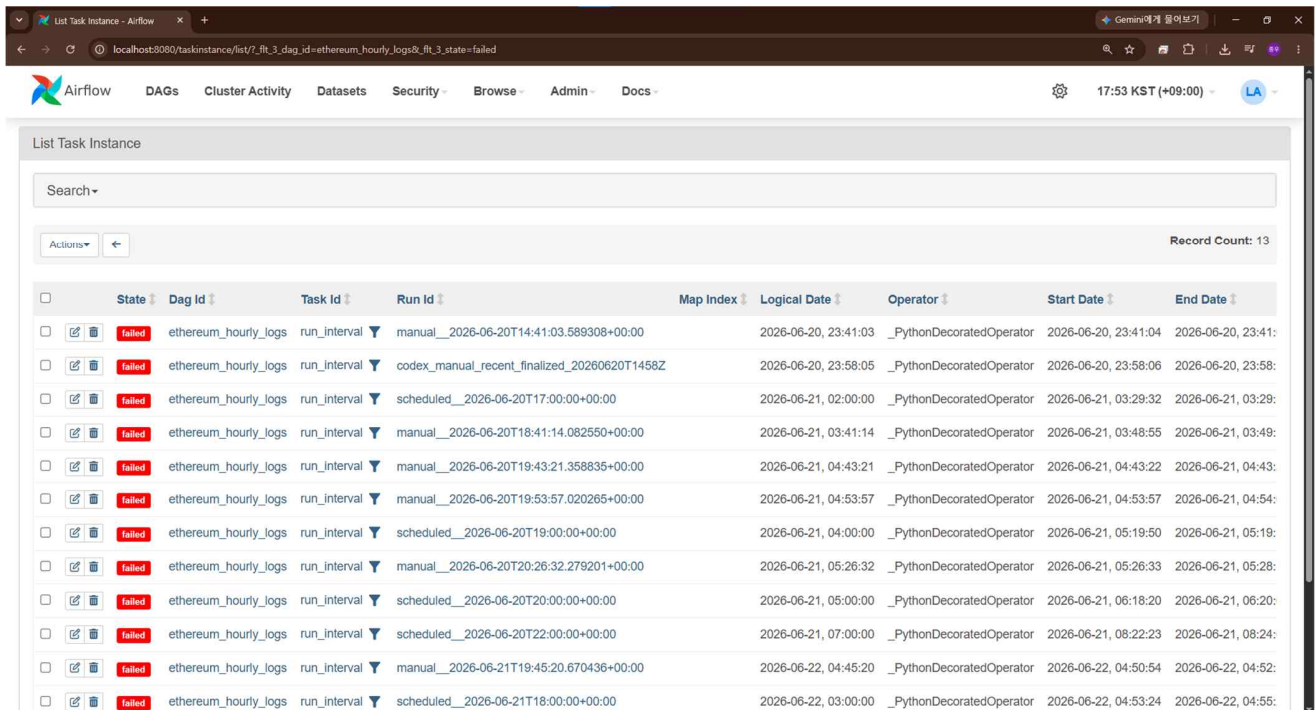
DAG 등록, @hourly schedule, tags 와 summary run counts 를 보이는 UI 보조 증거. data/imgs/task_02_01_image.png

Figure 2. Airflow DAG grid



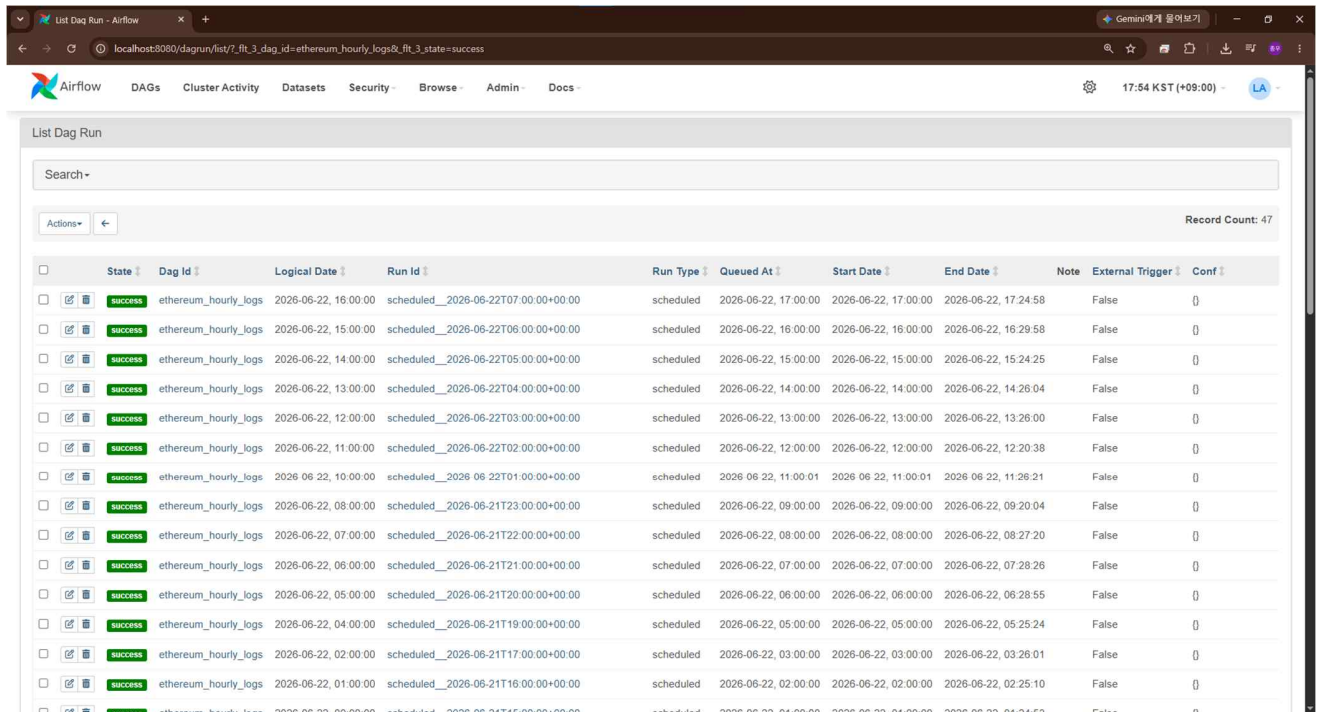
scheduled/manual run history와 success/failed 혼재를 보이는 UI 보조 증거. data/imgs/task_02_02_image.png

Figure 3. Failed task instance history



실패 task record를 재시도·디버깅 대상이 존재함을 보이는 UI 보조 증거(현 Debugging-fixed 된 사항). data/imgs/task_02_03_image.png

Figure 4. Successful DAG run history



The screenshot displays the Airflow web interface for the 'List Dag Run' page. The URL is localhost:8080/dagrun/list/?_flt_3_dag_id=ethereum_hourly_logs&_flt_3_state=success. The interface shows a table of DAG runs with columns: State, Dag Id, Logical Date, Run Id, Run Type, Queued At, Start Date, End Date, Note, External Trigger, and Conf. All runs are in a 'success' state. The table lists 47 runs, each with a unique Run Id and a corresponding Logical Date and End Date. The 'Record Count: 47' is displayed at the top right of the table.

State	Dag Id	Logical Date	Run Id	Run Type	Queued At	Start Date	End Date	Note	External Trigger	Conf
success	ethereum_hourly_logs	2026-06-22, 16:00:00	scheduled__2026-06-22T07:00:00+00:00	scheduled	2026-06-22, 17:00:00	2026-06-22, 17:00:00	2026-06-22, 17:24:58		False	0
success	ethereum_hourly_logs	2026-06-22, 15:00:00	scheduled__2026-06-22T06:00:00+00:00	scheduled	2026-06-22, 16:00:00	2026-06-22, 16:00:00	2026-06-22, 16:29:58		False	0
success	ethereum_hourly_logs	2026-06-22, 14:00:00	scheduled__2026-06-22T05:00:00+00:00	scheduled	2026-06-22, 15:00:00	2026-06-22, 15:00:00	2026-06-22, 15:24:25		False	0
success	ethereum_hourly_logs	2026-06-22, 13:00:00	scheduled__2026-06-22T04:00:00+00:00	scheduled	2026-06-22, 14:00:00	2026-06-22, 14:00:00	2026-06-22, 14:26:04		False	0
success	ethereum_hourly_logs	2026-06-22, 12:00:00	scheduled__2026-06-22T03:00:00+00:00	scheduled	2026-06-22, 13:00:00	2026-06-22, 13:00:00	2026-06-22, 13:26:00		False	0
success	ethereum_hourly_logs	2026-06-22, 11:00:00	scheduled__2026-06-22T02:00:00+00:00	scheduled	2026-06-22, 12:00:00	2026-06-22, 12:00:00	2026-06-22, 12:20:38		False	0
success	ethereum_hourly_logs	2026-06-22, 10:00:00	scheduled__2026-06-22T01:00:00+00:00	scheduled	2026-06-22, 11:00:01	2026-06-22, 11:00:01	2026-06-22, 11:26:21		False	0
success	ethereum_hourly_logs	2026-06-22, 08:00:00	scheduled__2026-06-21T23:00:00+00:00	scheduled	2026-06-22, 09:00:00	2026-06-22, 09:00:00	2026-06-22, 09:20:04		False	0
success	ethereum_hourly_logs	2026-06-22, 07:00:00	scheduled__2026-06-21T22:00:00+00:00	scheduled	2026-06-22, 08:00:00	2026-06-22, 08:00:00	2026-06-22, 08:27:20		False	0
success	ethereum_hourly_logs	2026-06-22, 06:00:00	scheduled__2026-06-21T21:00:00+00:00	scheduled	2026-06-22, 07:00:00	2026-06-22, 07:00:00	2026-06-22, 07:28:26		False	0
success	ethereum_hourly_logs	2026-06-22, 05:00:00	scheduled__2026-06-21T20:00:00+00:00	scheduled	2026-06-22, 06:00:00	2026-06-22, 06:00:00	2026-06-22, 06:28:55		False	0
success	ethereum_hourly_logs	2026-06-22, 04:00:00	scheduled__2026-06-21T19:00:00+00:00	scheduled	2026-06-22, 05:00:00	2026-06-22, 05:00:00	2026-06-22, 05:25:24		False	0
success	ethereum_hourly_logs	2026-06-22, 02:00:00	scheduled__2026-06-21T17:00:00+00:00	scheduled	2026-06-22, 03:00:00	2026-06-22, 03:00:00	2026-06-22, 03:26:01		False	0
success	ethereum_hourly_logs	2026-06-22, 01:00:00	scheduled__2026-06-21T16:00:00+00:00	scheduled	2026-06-22, 02:00:00	2026-06-22, 02:00:00	2026-06-22, 02:25:10		False	0
success	ethereum_hourly_logs	2026-06-22, 00:00:00	scheduled__2026-06-21T15:00:00+00:00	scheduled	2026-06-22, 01:00:00	2026-06-22, 01:00:00	2026-06-22, 01:24:53		False	0

성공 run list 를 보이는 UI 보조 증거. raw/dbt result 는 task log 와 Delta/DuckDB 직접 조회로 대조함. data/imgs/task_02_04_image.png

6. 미해결 항목, 한계 및 재개 전략

ID	미해결 또는 제한	현재 시도 / 완료 범위	영향	재개 전략 / 우선순위
L-01	Canonical reorg replacement	finality buffer 와 raw `block_hash` 보존. common-ancestor replacement fixture 미구현.	fork 이후 raw/silver/gold bounded rebuild 를 E2E 로 입증하지 못함.	synthetic fork fixture + chain revision/audit table + affected date rebuild. P1.
L-02	Full-history backfill	data interval/conf window 구조와 1-hour replay path 구현. full history 는 실행하지 않음.	provider 비용·시간·coverage 는 미검증.	provider capability test; adaptive chunking/backoff metrics; bounded batch plan. P1.
L-03	Provider SLA / alerting / secret rotation	local Docker run history 및 retry 만 확인.	무중단 운영·incident response 능력을 주장할 수 없음.	metrics/alert channel, rotation runbook, long-run SLO test. P2.
L-04	Base Delta schema drift	notebook 04 로 stale fields/legacy staging relation 노출.	base path 를 latest implementation evidence 로 사용하면 오판 위험.	base artifact archive/remove or migration; canonical v2 path document lock. P0.
L-05	Token metadata provenance	configured USDT contract and 6 decimals policy 만 사용.	contract metadata validity/decimals on-chain verification 없음.	versioned token dimension + `decimals()` validation. P2.
L-06	Private GitHub release state	README/checklist 에 final action 을 남김.	remote push / collaborator access 는 repo filesystem 만으로 검증 불가.	push + `dev@cryptoquant.com` invite + external access confirmation. P0.

6.1 설계상 트레이드오프

선택	얻는 것	잃는 것	선택 이유
10-block chunk	provider result/range cap 에 보수적.	RPC 호출 수 증가.	free-tier 실패율과 deterministic chunking 우선.

선택	얻는 것	잃는 것	선택 이유
4 RPS / concurrency 1	rate-limit pressure 및 local single-writer race 완화.	throughput 제한.	과제 규모·무료 provider 조건에 적합.
finalized head guard	미확정 구간 raw insert 를 피하고 reorg risk 감소.	recent interval latency 증가.	correctness 를 latest block freshness 보다 우선.
Delta insert-if-not-exists	replay/backfill duplicate 방지.	existing key lookup 비용.	natural-key correctness 가 primary invariant.
single dbt selector task	DAG 수정 없는 model expansion, simple orchestration.	model-level Airflow observability 낮음.	dbt graph 를 source of truth 로 유지.
Task1 policy metric	정의와 재현성 명확.	proprietary product metric 과 차이.	확인 불가 내부 algorithm 복제를 주장하지 않기 위함.

7. 실행 방법 및 제출 체크리스트

7.1 Repository 실행 방법 요약

단계	명령 또는 작업	예상 결과	근거 위치
1	Repository clone 후 `.env.example`를 기준으로 non-secret config 와 `ETH_RPC_URL`을 local env 에 주입.	secret 이 repository 에 없고 provider config validation 통과.	README setup / provider.py.
2	`docker compose up -d` 또는 project compose stack 실행.	Airflow services 및 workspace-dev runtime available.	README execution guide.
3	Airflow `ethereum_hourly_logs` scheduled/manual run 또는 conf/data interval trigger.	UTC 1-hour range 의 raw log→Delta→dbt pipeline.	airflow DAG / docs 04-05.
4	fixture build: `dbt build --select tag:ethereum_hourly --vars window --no-partial-parse`.	models/tests combined pass; documented result PASS=43.	docs/06 validation.
5	`python -m pytest -q`; notebooks 03/04 output review.	49 tests pass; replay evidence and freshness boundary verified.	docs/06; src/notebooks/.

7.2 제출 체크리스트

상태	체크 항목
VERIFIED	Private GitHub Repository 에 최종 코드·문서·구성 파일 push. remote SHA 와 branch 상태 확인 필요.
VERIFIED	`dev@cryptoquant.com` Collaborator 초대 및 접근 가능 여부 확인 필요.
VERIFIED	README 에 실행 방법, 환경 변수 설명, 설계 경계, 디렉터리/증거 경로가 기록됨.
VERIFIED	tracked-file secret-like token scan 은 clean. Git history 및 screenshot/log final manual inspection 은 제출 직전 재확인 필요.
VERIFIED	Task 1 의 data contract, policy, pseudo SQL, dummy output, daily batch, reorg design 이 문서화됨.
VERIFIED	Task 2 의 Airflow, Delta, dbt models, idempotency, retry/backfill boundary, image/log/metadata evidence 가 연결됨.
VERIFIED	신규 dbt quality summary 가 DAG 수정 없이 selector/ref graph 에 포함되는 증거가 있음.
VERIFIED	AI 활용 목적·대표 프롬프트·검증 방식·미반영 경계가 기록됨.
VERIFIED	미구현 사항을 삭제하지 않고 limitation/restart plan 으로 기록함.
VERIFIED	최종 DOCX/PDF export 후 표·code block·page break render QA 필요. 해당 DOCX 문서는 render QA 후 산출함.

7.3 최종 요약

항목	결론
Task 1	Bitcoin Velocity 를 단순 비율이 아니라 chain state-volume definition-supply policy-version-quality gate 가 묶인 data product contract 로 설계했다. production metric 재현이나 Bitcoin execution 은 주장하지 않음.
Task 2	Airflow data interval, finalized range, chunked `eth_getLogs`, idempotent Delta write, dbt selector graph, fixture/live evidence 를 연결했다. latest scheduled evidence 는 6.08M raw rows 와 downstream materialization 을 지시함.

항목	결론
가장 중요한 설계 결정	raw natural key `(chain_id, transaction_hash, log_index)`, UTC `[start,end)` time contract, finalized head guard, policy-versioned Task1 metric, selector-based dbt build.
남은 리스크	canonical reorg replacement, provider SLA/alerting/rotation, full-history backfill, stale base artifacts, GitHub release/invite action.
포지션 정합성	데이터가 들어오는 순간부터 raw contract, retry/backfill, idempotency, downstream modeling, run evidence, limitation disclosure 까지 연결하는 데이터 플랫폼 사고방식을 보인다. 단, production-scale operation 은 검증하지 않았음을 명확히 분리함.

부록 A. 대표 프롬프트 원문형 요약

Prompt ID	도구	목적	핵심 발췌	검증 결과
P-01	GPT	리팩토링 범위 설정	“현재 Repository 의 Python 코드와 SQL 또는 dbt SQL 을 리팩토링하고, 코드 주석·Markdown 문서·체크리스트·요구사항 추적표까지 함께 최신화하라.”	compileall, pytest, dbt build, Markdown link check 후 반영.
P-02	GPT	AI 투명성/책임 경계	“AI 활용 투명성 원칙을 지키고, AI 가 확장한 검토 범위와 사용자가 책임진 최종 판단을 분리해서 보여줘.”	README, docs/09, traceability matrix 에 반영.
P-03	Codex	Notebook 검증 보조	“`~/workspace/src/notebooks` 하위 주피터 노트북 파일에서 내용을 확인하고, Python source code 검증 및 데이터 확인에 주안점을 두어 정리해줘.”	nbclient output 저장 및 JSON parse 후 03/04 상태 분리.
P-04	Codex	Image evidence 연결	“`~/workspace/data/imgs` 하위 이미지 파일을 분석하고 Markdown 문서에 실행 증거로 명시해줘.”	UI screenshot 을 task log·Delta/DuckDB 와 대조하여 partial evidence 로 제한.
P-05	GPT/Codex	재판정	“미검증과 체크 안 된 사항들을 로그와 메타데이터로 재고하고, 과제-1 의 이해당사자별 타당성 별도 검증해서 문서 최신화해줘.”	logs/storage/Task1 static scan 으로 status refresh.

부록 B. 더미 데이터와 검증 쿼리 / 출력

```
-- Notebook 03 fixture replay assertion summary
raw_log_count = 2
unique_natural_key_count = 1
normalized_log_count = 2
invalid_log_count = 0
decoded_transfer_count_after_key_dedupe = 1
first_inserted_row_count = 1
second_inserted_row_count = 0
row_count_after_second_write = 1
duplicate_natural_key_count = 0

-- Natural key invariant
SELECT chain_id, transaction_hash, log_index, COUNT(*) AS cnt
FROM ethereum_logs
GROUP BY 1,2,3
HAVING COUNT(*) > 1; -- expected: 0 rows

-- Task 1 illustrative 3-day fixture result
2026-06-03 velocity
= (100.0 + 98.0 + 94.5) / 100.0
= 2.925

-- Task 1 production rule: do not coerce missing source day to zero.
-- date spine must have 365 calendar rows, 365 volume source rows,
-- and 365 supply source rows before publish.
```


부록 C. 저장소 구조 및 주요 파일 맵

```
[repository-root]
├── README.md
├── docs/
│   ├── 04_execution_guide.md
│   ├── 06_validation_evidence.md
│   ├── 09_ai_usage_transparency_and_validation.md
│   ├── 10_requirement_traceability_matrix.md
│   ├── task_01_bitcoin_velocity/
│   └── task_02_ethereum_log_pipeline/
├── airflow/dags/ethereum_hourly_logs.py
├── src/cryptoquant_pipeline/
│   ├── block_range.py      # UTC interval + finality guard + binary search
│   ├── chunking.py         # 10-block invariant
│   ├── rpc_client.py       # JSON-RPC retry/rate limit classification
│   ├── log_normalizer.py   # raw event normalization / uint256 contract
│   ├── delta_writer.py     # natural-key insert-if-not-exists
│   ├── dbt_runner.py       # tag:ethereum_hourly build boundary
│   └── quality_checks.py   # raw/analytics quality metrics
├── src/notebooks/
│   ├── 03_fixture_etl_replay_idempotency_validation.ipynb
│   └── 04_accumulated_pipeline_data_freshness_validation.ipynb
├── dbt/models/ and dbt/tests/
├── tests/
├── data/imgs/task_02_01_image.png ... task_02_04_image.png
├── docker-compose.yaml
└── .env.example # secret-free sample
```

증거 원천 목록

본 보고서는 `README.md`, `docs/06\_validation\_evidence.md`,
`docs/09\_ai\_usage\_transparency\_and\_validation.md`, `docs/10\_requirement\_traceability\_matrix.md`,
`docs/11\_refactoring\_report.md`, Task 1/Task 2 상세 Markdown, `src/cryptoquant\_pipeline/\*.py`,
`src/notebooks/03\_\*` 및 `04\_\*` 저장 output, `data/imgs/task\_02\_01..04\_image.png`을 기준으로 작성했다. 이
보고서는 해당 마크다운 원천 문서에 기록된 실행 결과를 재구성한 것이며, report 작성 중 raw provider
endpoint 나 secret 을 기록하지 않음.